北京大学
PEKING UNIVERSITY

JIT-AGENT: SCALING HARNESS INTELLIGENCE VIA JUST-IN-TIME HARNESS EVOLUTION

Core Contributors: Guibin Zhang, Leo Lu, Fangzhou Xie
Affiliations: National University of Singapore, EverMind AI, Nanyang Technological University

汇报人：耿存真　　**汇报日期**：2026/09/30

目录

CONTENTS

1 基本信息

2 研究背景

3 研究思路

4 研究结果

5 结论与讨论

6 进一步讨论

1 基本信息

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

论文信息	
标题	JIT-AGENT: Scaling Harness Intelligence via Just-in-Time Harness Evolution
形式	技术报告（arXiv preprint），尚无正式会议/期刊录用信息
版本	arXiv:2608.25593v2 [cs.CL]，2026 年 9 月 3 日
核心贡献	Guibin Zhang、Leo Lu、Fangzhou Xie
通讯作者	Wangchunshu Zhou、Chuanrui Hu、Yafeng Deng、Shuicheng Yan
机构	新加坡国立大学 (NUS)、EverMind AI、南洋理工大学 (NTU)
开源	`github.com/bingreeky/JIT`

Fork 58 Star 514

2 现有方法与局限

方法	代表工作 / 做法	共同问题
Harness Engineering and Modularization	Claude Code、Codex、Hermes 等运行时，模块化接口	领域不同，但共享 Memory / Planning / Action / Capability 的操作层
Harness 优化	diagnose–propose–validate 循环，优化 prompt/工作流/harness 代码	**优化的是持久工件(H→H′→H′′)**
递归、层次化、在线优化变体	改进循环的 recursive/hierarchical/online 版本	相对新任务实例仍是 AOT
JIT-Agent（本文）	把搜索摊销进训练好的生成器，逐任务即时合成	**范式切换：AOT → JIT**

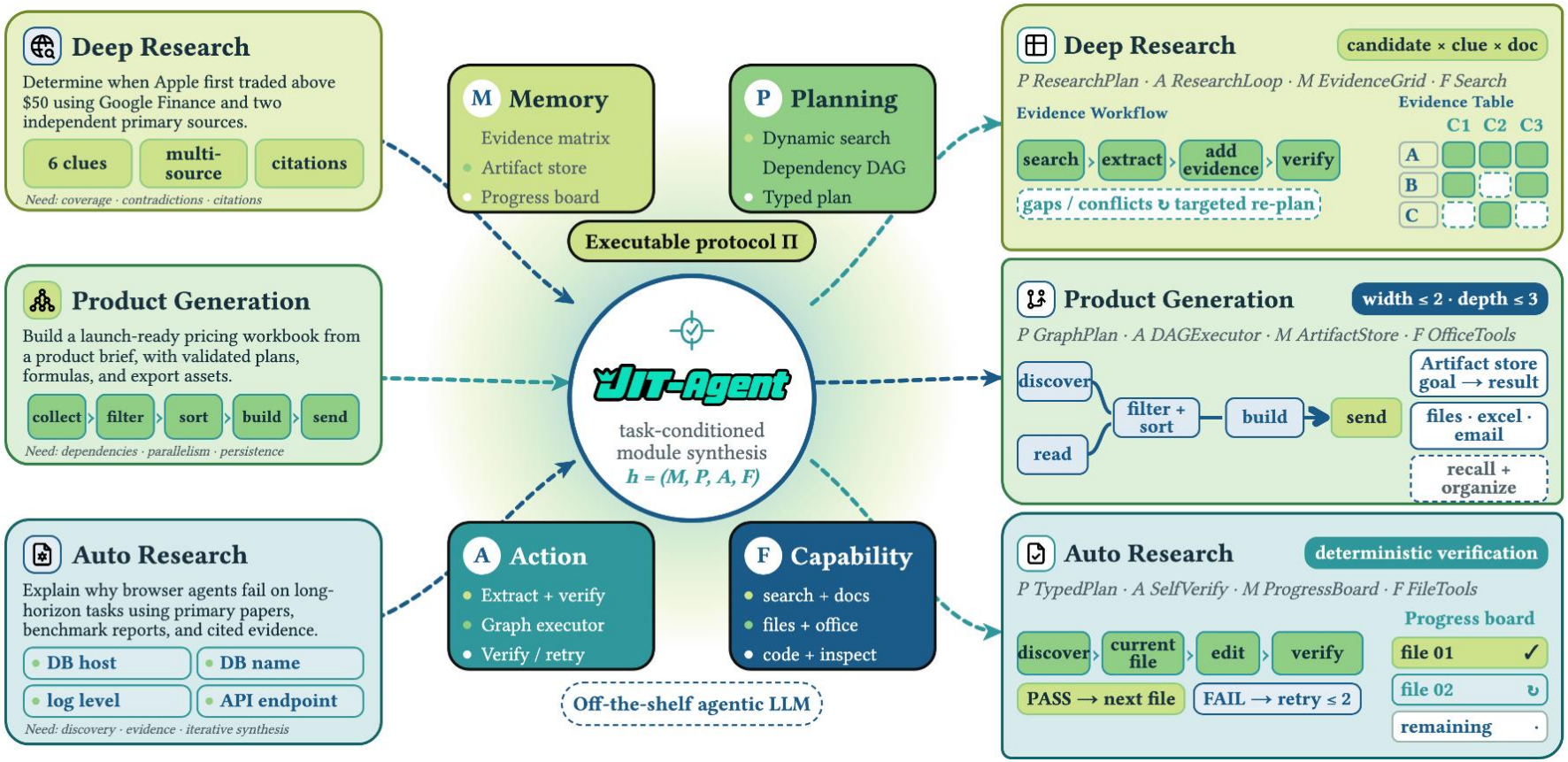
Table 4: **Harness optimization paradigms.** *Construction* distinguishes harnesses found by ahead-of-time search, ahead-of-time harnesses subsequently edited with test-time feedback, and harnesses generated just in time for each task. *Instance synthesis*, *Harness model*, *Learned repair*, and *Online evolution* indicate whether a method directly synthesizes an instance-specific harness, trains the generator, learns repair from failed execution trajectories, and continues improving after deployment, respectively.

Method	Construction	Instance synthesis	Harness model	Learned repair	Online evolution
AutoHarness (Lou et al., 2026)	AOT (search)	✗	✗	✗	✗
Meta-Harness (Lee et al., 2026c)	AOT (search)	✗	✗	✗	✗
AHE (Lin et al., 2026)	AOT (search)	✗	✗	✗	✗
Adaptive AH (Liu et al., 2026b)	AOT (test-time editing)	✗	✗	✗	✓
TTHE (Nie et al., 2026)	AOT (test-time editing)	✗	✗	✗	✓
RHI (Lee et al., 2026a)	AOT (test-time editing)	✗	✗	✗	✓
Harness-R1 (Shao et al., 2026)	AOT (test-time editing)	✗	✓	✓	✓
JIT-Agent (ours)	**JIT**	✓	✓	✓	✓

Harness 不仅具有领域/系统差异，还需要根据具体任务实例进行条件化 → JIT-Agent 在任务到来时即时生成 task-conditioned Harness

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

3 JIT-Agent 总体框架：Model-as-a-Harness



输入：任务规格 τ + 固定协议 Π + 可执行工具/技能注册表 C_τ + 少量检索到的历史 harness 上下文 E_τ

输出：一份**可直接执行**的任务专属 harness h，它包裹一个 off-the-shelf 的 agentic LLM 去执行

论文 Figure 1。JIT-Agent 对任务**实例化**（而非简单拼接）四个模块；不同的任务结构因此诱导出**完全不同的可执行协议与状态组织**。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

3 统一代码库：四模块协议与 HarnessFactory

把 Harness 因子化为四元组

模块	回答的问题	产出
M Memory	之前的历史如何压缩？哪些留给模型看？	视图 v_t
P Planning	要不要先列计划？做偏了要不要重规划？	局部指令 d_t
F Capability	这一步暴露哪些工具 / 技能 / MCP？	能力集 C_t
A Action	输出什么格式？怎么解析？何时结束？	状态 s_{t+1} 与动作 e_t

$$
\begin{aligned}
\mathbf{v}_t &= \mathbf{M}(\boldsymbol{\xi}_{<t}, \mathbf{s}_t) \in \mathcal{V}, && \text{history} \to \text{view},\\
\mathbf{d}_t &= \mathbf{P}(\boldsymbol{\tau}, \mathbf{s}_t, \mathbf{v}_t) \in \mathcal{D}_{\text{dir}}, && \text{view} \to \text{local directive},\\
\mathcal{C}_t &= \mathbf{F}(\mathcal{C}_\tau, \mathbf{s}_t, \mathbf{v}_t, \mathbf{d}_t) \subseteq \mathcal{C}_\tau, && \text{directive-conditioned capability orchestration},\\
(\mathbf{s}_{t+1}, e_t) &= \mathbf{A}(\mathbf{s}_t, \boldsymbol{\tau}, \mathbf{v}_t, \mathbf{d}_t, \mathcal{C}_t) \in \mathcal{S} \times \mathcal{A}, && \text{control update and action emission},
\end{aligned}
$$

生成空间是嵌套的：

$$\mathcal{G} \supseteq \mathcal{H}_{syn} \supseteq \mathcal{H}_{\Pi} \supseteq \mathcal{H}_{\Pi}^{exec}$$

四元组是**概念上的分解**；运行时真正的依赖顺序是

$$M \to P \to F \to A$$

记忆先产出视图，规划才能形成指令，能力编排依赖指令，动作最后发射。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

3 HarnessFactory

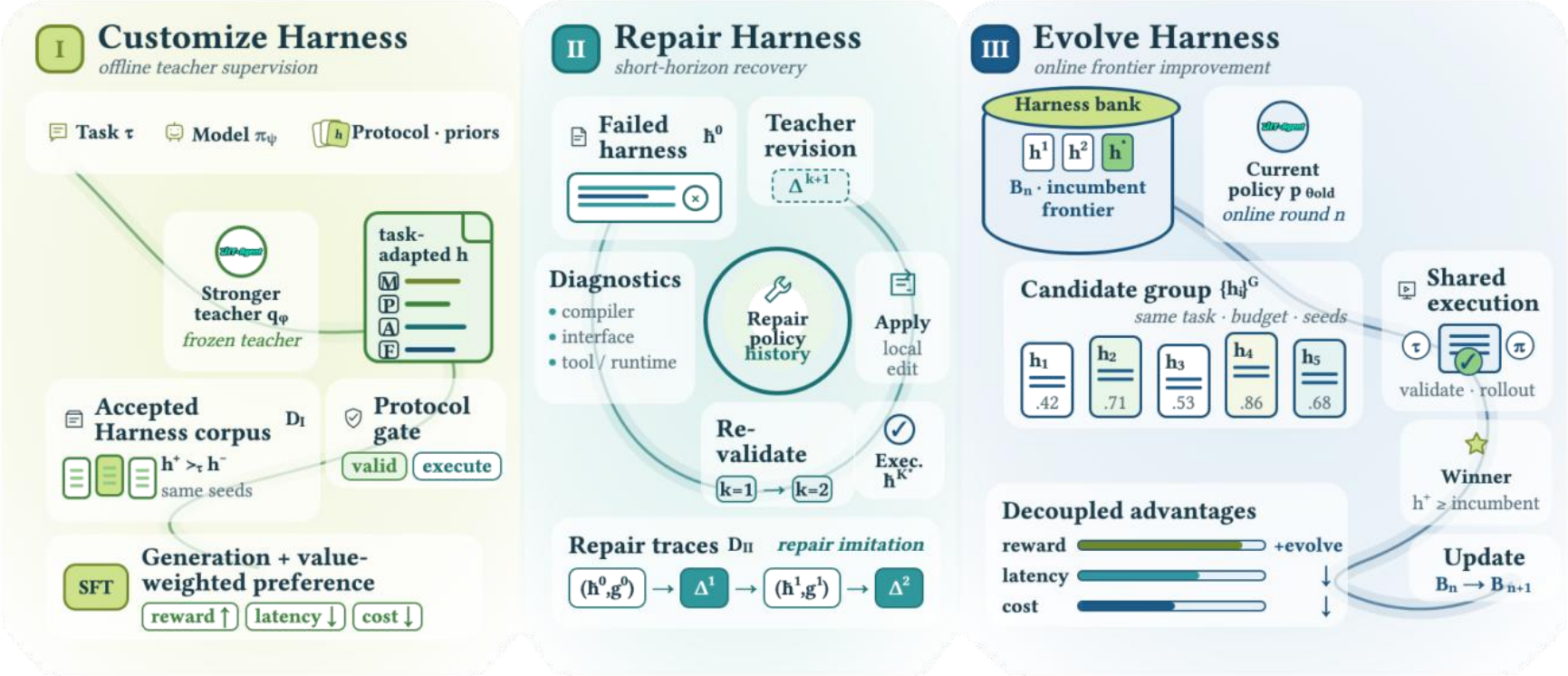
表 1: 种子库 $\mathcal{B}_0$ ： 13 个手工编写的测试用例，实例化了四模块协议 $\mathbf{\Pi}$ 。每一行是一个完整的测试用例；列按照内存、规划、动作和能力协调的概念顺序排列。

Harness	♣ Memory $M \in \mathfrak{M}$	♦ Planning $P \in \mathfrak{P}$	♥ Action $A \in \mathfrak{A}$	♠ Capability Orchestration $F \in \mathfrak{F}$
ReAct [69]	FullHistory	No explicit planner	ReAct	Full registry
Plan-and-Execute [14]	FullHistory	Linear roadmap	ReAct	Full registry
ReSum [63]	ReSum memory	No explicit planner	ReAct	Full registry
Flash-Searcher [51]	FullHistory	DAG planning	ReAct	Full registry
GAM [65]	GAM retrieval	DAG	ReAct	Full registry
MemoBrain [50]	Reasoning graph	No explicit planner	Marker-guided execution	Full registry
AggAgent [29]	Isolated rollout histories	No explicit planner	Multi-rollout aggregation	Full registry
OAgent [80]	Coordinator history	No explicit planner	Ensemble voting	Full registry
AgentFold [70]	AgentFold memory	DAG	ReActFold	Full registry
HiAgent [20]	Hierarchical memory	No explicit planner	ReAct	Full registry
DeepAgent [30]	Three-tier memory	No explicit planner	Marker-guided execution	Tool search
ROMA [1]	Context isolation	Atomizer + DAG	Recursive execution	Full registry
AOrchestra [53]	Context isolation	Atomizer + DAG	Recursive execution	Agent Delegation

每一行是一个**完整 harness**；四列恰好对应记忆 / 规划 / 动作 / 能力编排

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

3 三阶段训练：定制 → 修复 → 进化



Stage I （Customization，定制）：更强教师合成协议合规 harness → SFT + 价值加权偏好（reward 提升且 latency/cost 不劣才保留） 针对不同任务生成不同 Harness

Stage II （Repair，修复）：失败 harness + 结构化诊断 → 只保留 ≤2 轮内可修复的轨迹，模仿学习短周期修复 Harness 出错后能够修复

Stage III （Evolution，Evo-GDPO）：在线超越 archive frontier；reward/latency/cost 三通道解耦归一，reward 主导 根据执行反馈产生更强 Harness

三阶段分别对应 harness intelligence 的三个性质：**自适应 / 可靠 / 可演化**

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

3 Stage I 定制 Adaptivity

数据准备

用**冻结的、更强的 teacher** q_ϕ 在固定协议下合成任务适配的 harness。

- 每个任务的 E_τ 含 **3 个从任务类型匹配**子集 $B_0^{(d(\tau))}$ 采样的脚手架
- teacher 的生成只有在通过**协议校验 + 执行检查**之后才被保留
- 训练任务来自已有 agentic benchmark 与环境，并补充合成任务

训练两小步

(1) SFT：直接教" 任务 + 参考上下文 → 协议合规 harness" 的映射。

(2) DPO：协议合规的 harness 之间质量与效率仍可能差很多，于是同一骨干、同一评测种子下比较候选。

效果更好
速度不能更慢
成本不能更高
并且速度或成本至少有一个真的更好

偏好判据与目标函数（论文 Eq. 4）

偏好只在「reward 提升 ∧ 两个效率轴都不退化 ∧ 至少一个效率严格改善」时才保留：

$$h^+ \succ_\tau h^- \iff r^+ > r^- \ \wedge\ \ell^+ \le \ell^- \ \wedge\ \kappa^+ \le \kappa^- \ \wedge\ (\ell^+ < \ell^- \vee \kappa^+ < \kappa^-)$$

$$\Delta_{val}(\tau; h^+, h^-) = \alpha_r(r^+ - r^-) + \alpha_\ell[\ell^- - \ell^+]_+ + \alpha_\kappa[\kappa^- - \kappa^+]_+$$

其中 $(r^\pm, \ell^\pm, \kappa^\pm)$ 是重复 rollout 的均值，$[x]_+ = \max(x, 0)$。用 Δ_{val} 加权一个标准 DPO 目标，锚定在 Stage-I 的 SFT checkpoint 上。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

3 Stage II 修复 Reliability

问题

Stage I 只优化了**协议合规**，但仍有生成的 harness 通不过静态或运行时校验。

做法：不丢弃，而是把失败变成监督

1. 给每个失败的 harness $\hat{h}^{(0)}$ 配一份**结构化诊断** $g^{(0)}$——记录编译错误、接口不匹配、工具调用失败、运行时异常；
2. 第 k 轮修复：teacher 从**补丁空间** $\mathcal{P}$ 提出结构化修订 $\Delta^{(k+1)}$，`Apply` 确定性地更新 harness，再校验产生下一个诊断；
3. **只保留两轮内变得可执行的轨迹**：

$$\hat{h}^{(k+1)} = \mathrm{Apply}\big(\hat{h}^{(k)}, \Delta^{(k+1)}\big),\ k \in \{0,1\}, \qquad K^\star = \min\{k \in \{1,2\} : \mathrm{Valid}_\Pi(\hat{h}^{(k)}) = 1\}$$

训练与要点

对成功的修订 $\{\Delta^{\star(k+1)}\}$ 做 teacher-forced 模仿，条件在 c_τ 与**完整的 harness–诊断历史**上。

因为 $K^\star \le 2$，模型学到的是**短视界**策略：用**少量高杠杆的修订**恢复可执行，而不是推倒重写。

监督语料由此聚焦在**现实中局部可恢复的失败**，而不是需要整体重设计的失败。

3 Stage III Evo-GDPO evolvebility

把”测试时 harness 演化”变成一种**可训练**的能力

在线第 n 轮：采样任务 $\tau \to$ 从当前 bank B_n 检索高质量设计 $E_{\tau,n} \to$ 构成 $c_{\tau,n} = (\tau, \Pi, C_\tau, E_{\tau,n})$。
模型从当前 rollout policy **独立采样 $G > 1$ 个候选**；候选与检索到的设计在**同一冻结执行器、同一预算、同一种子**下评估。
检索到的最高 reward harness（同分比 latency，再比 cost）给出**在位者统计** (b_r, b_ℓ, b_κ)。

奖励设计：三个通道解耦（论文 Eq. 6）

$$R_i^{rew} = r_i + \lambda_{evo}\,[r_i - b_r]_+ \qquad R_i^{lat} = \mathbb{I}[r_i \geq b_r]\,[b_\ell - \bar{\ell}_i]_+ \qquad R_i^{cost} = \mathbb{I}[r_i \geq b_r]\,[b_\kappa - \bar{\kappa}_i]_+$$

读法：只有**超越在位者**才拿额外奖励；效率奖励**只有在 reward 不掉的前提下**才激活。

归一化与更新规则

三个通道**在候选组内各自归一化**后再加权，避免量纲互相淹没：
$A_i^\Sigma = w_{rew}A_i^{rew} + w_{lat}A_i^{lat} + w_{cost}A_i^{cost}$，且 $w_{rew} > w_{lat} + w_{cost}$（保证 reward 主导）。
再对 batch 归一化，用标准 clipped PPO + 对 Stage-II checkpoint 的 token 级 KL 惩罚。
bank 保守更新：候选必须**追平或超过当前 reward 前沿**，且**严格改善**至少一个前沿维度（reward / latency / cost）才被保留。
训练时反馈同时更新 θ 与 B_n；**部署时 θ 冻结**。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

4 实验设置

9 个 Benchmark，4 类任务（全部归一到 0–100，越高越好）

任务类型	Benchmark（指标）
Deep Research	BrowseComp-Plus（accuracy）、DeepSearchQA（answer F1）、xBench-DeepSearch（accuracy）
Daily Work	AgentIF-Oneday（归一化加权 rubric 分）、PinchBench（平均分）
Planning	DeepPlanning-Shopping（购物车匹配率）、DeepPlanning-Travel（复合约束分）
Workspace	OfficeBench、OdysseyBench（均为任务成功率）

Backbones

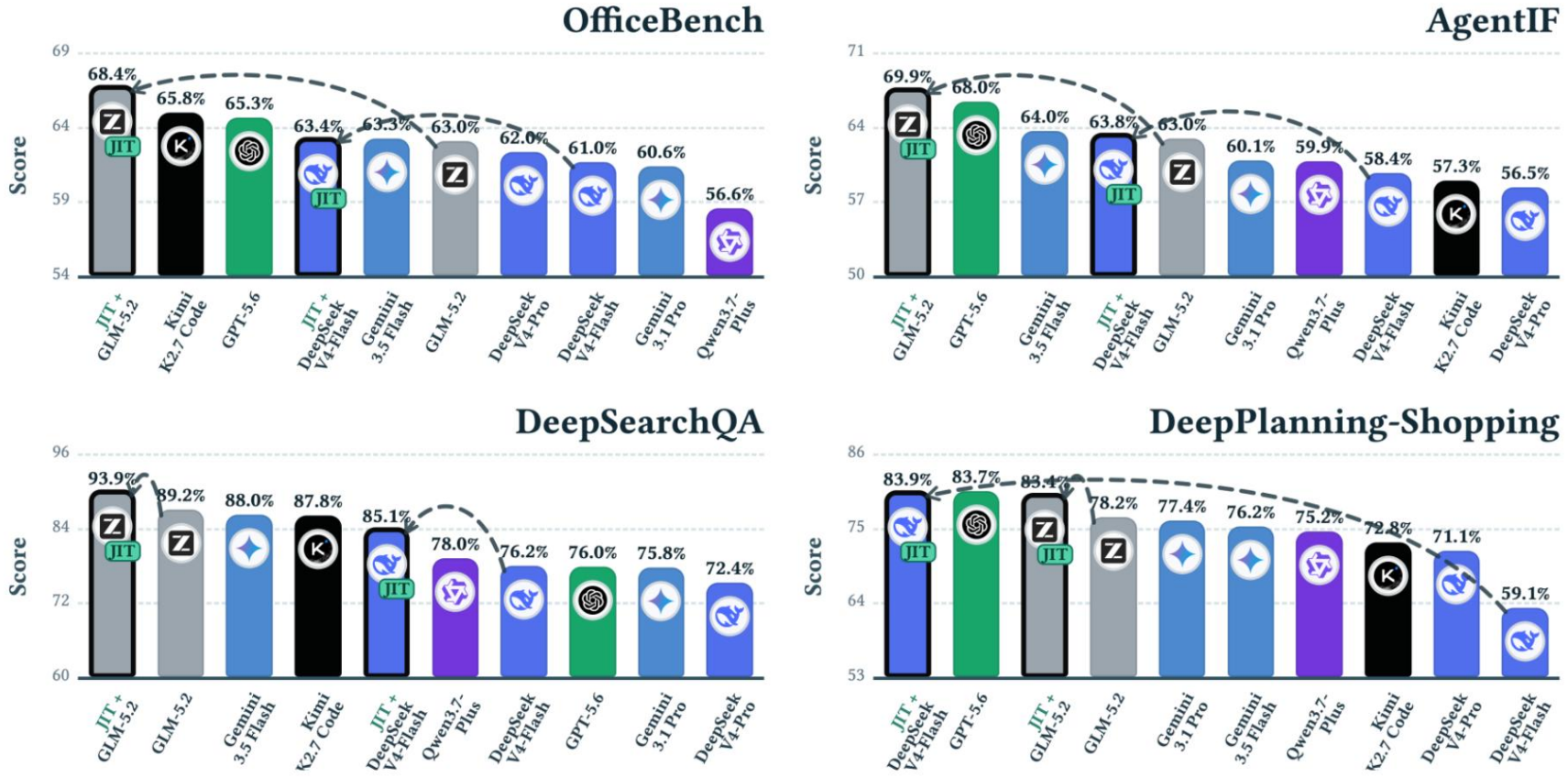
JIT-Agent 基于 **Qwen3.6-27B** 训练。
主力实例化 **GLM-5.2** 与 **DeepSeek-V4-Flash-Preview**。
泛化测试 Qwen3.6、Mimo-V2.5 系列。

两组基线

广度对比：Qwen3.7-Plus、GLM-5.2、DeepSeek-V4-Flash/Pro、Kimi K2.7 Code、GPT-5.6-Sol、Gemini 3.1 Pro / 3.5 Flash。
固定骨干对比：Claude Code、Codex、OpenCode、Hermes、NanoBot。
第二组的作用是把 harness 的贡献从模型权重里**隔离出来**。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

4 主实验结果：18/18 骨干-benchmark 对全部提升



- 替换默认 scaffold：GLM-5.2 均分 74.1→81.8（+7.7），DeepSeek-V4-Flash 66.7→75.5（+8.8）
- 开源骨干在 9 列中拿 8 个榜首：JIT+GLM-5.2 七列第一（DSQA 93.9、PinchBench 93.3、AgentIF 69.9）
- 最大单点增益 +24.8（Shopping 59.1→83.9）；DSQA 上 V4-Flash+JIT 85.1 反超 GPT-5.6 +9.1，OdysseyBench +4.3
- 平均超 DeepSeek-V4-Pro 8.7 分 —— 收益来自任务自适应 harness，而非更大的模型

4 主实验结果：18/18 骨干-benchmark 对全部提升

表 2: 九个 Agentic 基准上的主要结果。以 JIT-Agent 开头的行使用与对应原始版本相同的主干模型，但将其默认框架替换为 JIT 生成的框架。得分以 0 –100 尺度报告；数值越高越好。每列中的最高、第二高和第三高得分分别用浅绿色、黄绿色和黄色突出显示；最高得分还加粗显示。

Model	Deep Research			Daily Work		Planning		Workspace	
	BC+	DSQA	xBench	AgentIF	PinchBench	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.1	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	69.9
GPT-5.6	76.9	76.0	81.0	68.0	84.2	83.7	**84.9**	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT-Agent + GLM-5.2	**78.0**	**93.9**	**88.0**	**69.9**	**93.3**	83.4	83.0	**68.4**	**78.7**
JIT-Agent + DeepSeek-V4-Flash	74.0	85.1	82.0	63.8	92.9	**83.9**	61.3	63.4	73.0

- 替换默认 scaffold：GLM-5.2 均分 74.1→81.8（+7.7），DeepSeek-V4-Flash 66.7→75.5（+8.8）
- 开源骨干在 9 列中拿 8 个榜首：JIT+GLM-5.2 七列第一（DSQA 93.9、PinchBench 93.3、AgentIF 69.9）
- 最大单点增益 +24.8（Shopping 59.1→83.9）；DSQA 上 V4-Flash+JIT 85.1 反超 GPT-5.6 +9.1，OdysseyBench +4.3
- 平均超 DeepSeek-V4-Pro 8.7 分 —— 收益来自任务自适应 harness，而非更大的模型

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

4 对比固定 Harness：更强，且更便宜

表 3: 与先进智能体框架的受控对比。对于每个固定的骨干模型，我们报告任务性能（Perf.）、每例平均 token 消耗量（千单位）（#Tokens (K)）以及每例 API 成本（美元）。性能越高越好；token 消耗量和成本越低越好。同一骨干模型组内的最佳值以粗体标出。

Backbone	Harness	DeepSearchQA			xBench-DS			AgentIF		
		Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓
DeepSeek-V4-Flash	Claude Code	79.6	625	$0.088	75.0	559	$0.079	**66.9**	808	$0.114
	Codex	77.8	760	$0.107	70.0	680	$0.096	58.5	870	$0.123
	OpenCode	75.9	1,832	$0.258	65.0	1,157	$0.159	48.1	950	$0.135
	Hermes	69.9	1,157	$0.163	72.0	1,254	$0.177	60.3	1,000	$0.142
	NanoBot	80.4	924	$0.131	78.0	527	$0.075	53.1	1,034	$0.147
	JIT-Agent	**85.1**	**400**	**$0.066**	**82.0**	**212**	**$0.039**	63.8	**476**	**$0.097**
Qwen3.6-Flash	Claude Code	72.8	710	$0.140	58.0	650	$0.128	55.4	900	$0.177
	Codex	68.5	980	$0.193	52.0	874	$0.172	34.2	839	$0.170
	OpenCode	64.1	1,968	$0.384	44.0	1,300	$0.256	38.7	961	$0.217
	Hermes	60.8	1,319	$0.256	36.0	1,155	$0.227	49.7	1,184	$0.223
	NanoBot	**74.2**	892	$0.197	63.0	597	$0.119	43.5	950	$0.187
	JIT-Agent	70.3	**464**	**$0.095**	**70.0**	**300**	**$0.069**	**58.3**	**394**	**$0.078**

性能：6 个设置里 4 个最好，剩下 2 个也很接近

- DeepSeek-V4-Flash：比最强固定 harness 高 **+4.7**（DeepSearchQA）、**+4.0**（xBench-DS）
- Qwen3.6-Flash：高 **+7.0**（xBench-DS）、**+2.9**（AgentIF）
- 剩下 2 个设置只落后 **3.1** / **3.9** 分，但用的 token 少得多
- 作者由此指出：**没有任何单一固定 harness 能在所有任务上占优**

4 分析：成本–性能 Pareto 前沿

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

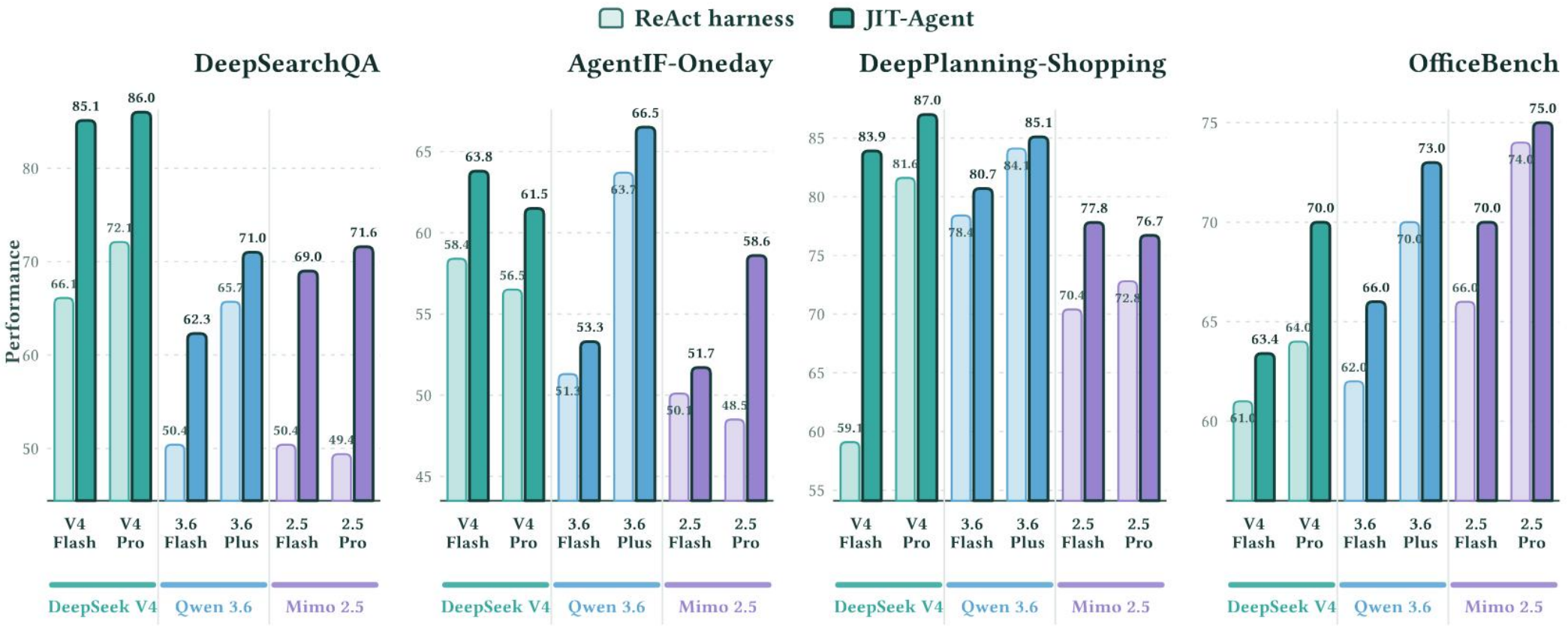


- **DeepSearchQA + DeepSeek-V4-Flash**：**85.1 分 @ $0.066**，相对最强固定 harness NanoBot **+4.7 分且降本 49.6%**（$0.131 → $0.066）。
- **DeepSearchQA + Qwen3.6-Flash**：**70.3 @ $0.095** vs NanoBot 74.2 @ $0.197——**降本 51.8%**，代价是 3.9 分。
- **AgentIF + Qwen3.6-Flash**：JIT **严格支配**所有固定 harness，最优分 55.4 → 58.3，最低成本 $0.170 → $0.078。

前沿是**任务相关**的，但 JIT 点**整体左移**——提升的是**推理效率**，而不是用更长轨迹买准确率。

4 分析：跨骨干泛化（24/24 全胜）

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论



论文 Figure 4。同一骨干在固定 ReAct harness 与 JIT harness 下的配对对比，六个骨干按模型家族分组。

6 个固定骨干（DeepSeek V4 / Qwen3.6 / Mimo-V2.5）× 4 个 benchmark，**24/24 全胜**，平均 +7.6；

家族级增益：DeepSeek **10.2** / Qwen **4.0** / Mimo **8.6**。DeepSearchQA 受益最大（平均 +15.2，含 Mimo-V2.5-Pro +22.2、DeepSeek-V4-Flash +19.0）。

说明 harness intelligence 是**跨模型家族与规模迁移**的，而不是在”补偿某一个骨干”。

4 分析2：生成的 Harness 长什么样

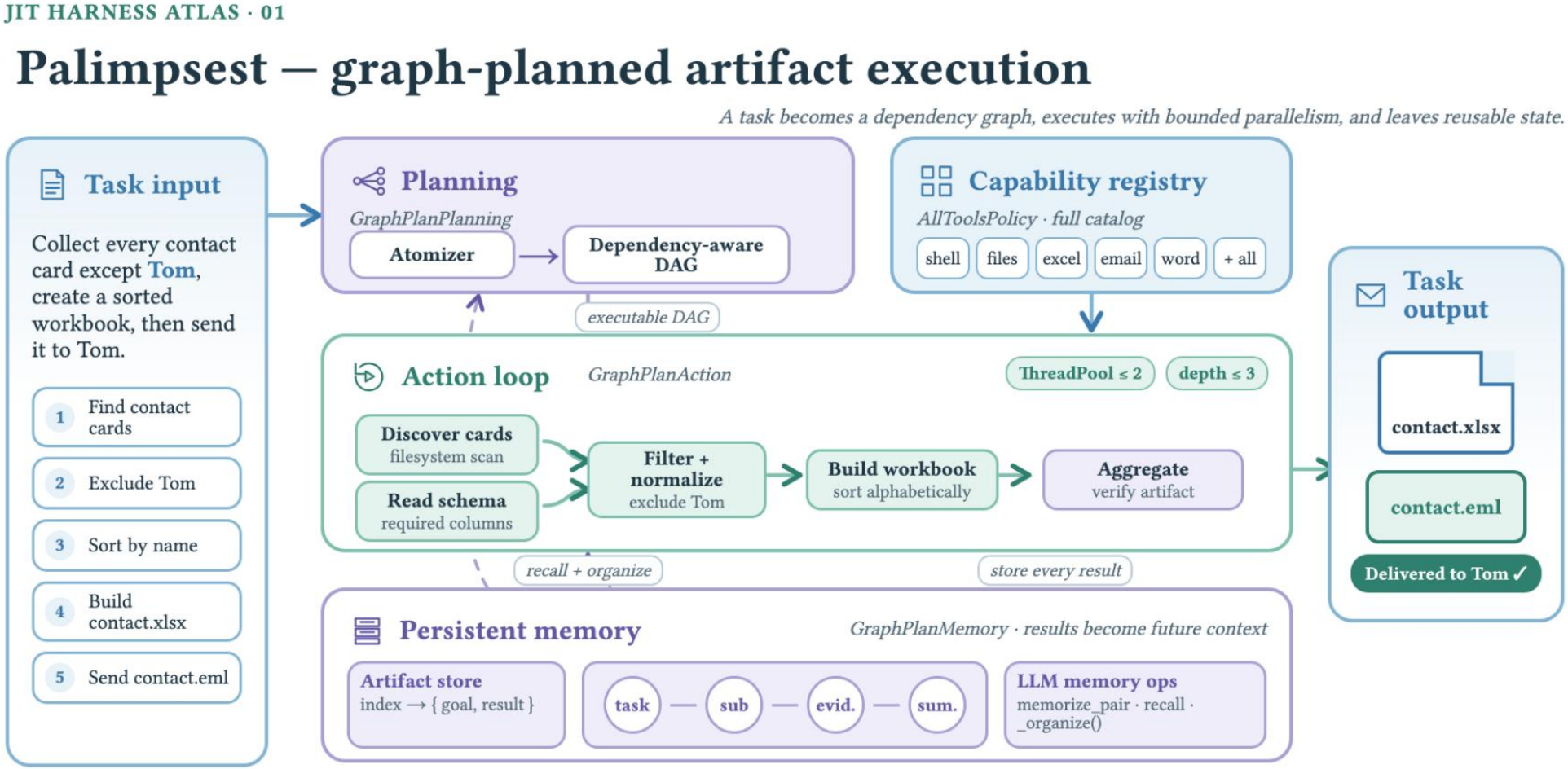


图 5: **Palimpsest：图规划的工件执行。** `GraphPlanPlanning` 将接触处理请求转换为有向无环图；`GraphPlanAction` 以有限宽度和深度执行该图，而 `GraphPlanMemory` 存储可重用的工件和推理状态。

5 结论与限制

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

论文自己的总结

JIT-Agent 是**第一个专为 just-in-time harness 生成而构建的模型**。
通过可组合的四模块协议表达 harness，并用「定制学习 + 修复监督 + Evo-GDPO」训练生成器，把 harness 构造从**手工的、提前的 artifact** 变成**一种可学习的能力**。

四类任务上的验证

- JIT 生成的 harness **持续强化**其底层骨干（18/18 配对提升）
- 能与**前沿闭源模型**和**先进固定 harness** 竞争（8/9 列第一）
- **改善成本–性能权衡**（成本全线最低，平均降低 36.0%）
- 能**跨模型家族迁移**（24/24 配对全胜）

最终主张

harness intelligence（调整”模型借以行动的运行脚手架”的能力）是一个**可训练、可迁移**的 agent 能力来源，与**模型 scaling 正交**。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

5 结论与限制

论文**没有单列 Limitations 章节**。以下七点是从它的实验设计与表述中读出的边界，供讨论。

1. **生成器本身并不通用**。JIT-Agent-27B 基于 Qwen3.6-27B 训练；跨到 GLM / DeepSeek / Mimo 的是**生成的 harness 可迁移**，不是**生成器可迁移**——论文没有报告”用别的模型当生成器”的结果。

2. **依赖一个更强的 teacher**。Stage I / II 的监督数据来自冻结的、更强的 teacher q_ϕ，teacher 的质量就是这套方法的上限。

3. **修复是有界的**。只保留两轮内可恢复的失败（$K^\star \le 2$）；需要整体重设计的失败被明确排除在监督之外——而这恰恰可能是最难的失败模式。

4. **评测用的是静态快照**。9 个 benchmark 中，两个 DeepPlanning 用的是**子集**；Figure 4 更明确使用 100 / 50 例子集，跨图比较需谨慎。

5. **演化只发生在 bank 上**。部署时 θ 冻结——模型参数不会在部署后继续学习，所谓”演化”是把经验存进 harness bank，而非参数更新。

6. **成本优势依赖定价**。$ per case 随供应商价格变动，Pareto 前沿会随之漂移；论文也没有明确说明 Table 3 的成本**是否计入 meta 模型生成 / 修复 harness 的开销**。

7. **与 AOT 优化方法没有正面对比**。Table 3 比的是成熟的生产 harness；对 Meta-Harness、Harness-R1 这类 AOT 优化方法只有 Table 4 的**结构性对比**——作者自己也声明它不是性能排名。

基本信息
研究背景
研究思路
研究结果
结论与限制
进一步讨论

6 进一步讨论：复现发现与启发

❶ **通用模型写 harness 会"写崩"。**
被淘汰的 rollout_0 里，memory.py 装进去的**根本不是代码，是模型的思考过程**（含一个裸露的 </think>）。这**精确命中**了仓库文档专门警告的坑：按标签切分时会在生成代码的**内部**切开，把 memory.py / planning.py 丢进 reasoning_content。
→ 说明"让通用模型输出格式严格、可直接执行的 5 文件 harness"是**一个很难的生成任务**——这正是论文要**专门训练一个模型**来解决的可靠性问题。

❷ **修复机制是必需品，不是锦上添花。**
如果没有修复循环，rollout_0 直接 0 分；修复把它救回 1.0。论文把它单列成 Stage II，是有道理的。

❸ **judge 的选择器判断是有依据的。**
judge 给出的理由（"会用 system.finish_task"）在后续执行中被验证为真。基于 LLM 的 harness 选择**不是随机猜测**。

❹ **非确定性确实存在。**
即使 benchmark 的 YAML 里配了 temperature: 0，同一个 case 两次运行的步数依然不同（4 步 vs 7 步）。做多轮实验时必须考虑这一点。

复现情况（officebench，免搜索 key，一个 LLM key 即可跑通）：

- **机制已跑通：固定 harness baseline 20 条 avg 0.85；完整 JIT 流程**
- **开放问题：JIT 与 AOT 混合（bank 演化如何平衡探索与利用）；跨任务 bank 复利与污染；生成代码的静态检查与沙箱边界**

北京大学
PEKING UNIVERSITY

汇报完毕，敬请老师同学批评指正！

汇报人：耿存真

汇报日期：2026/09/30